

cURL no Windows: testando APIs pelo terminal

1. O que é o cURL?

O **cURL** é uma ferramenta de linha de comando utilizada para transferir dados entre computadores utilizando diferentes protocolos de rede.

Para desenvolvimento de APIs, o protocolo que mais nos interessa é o HTTP/HTTPS.

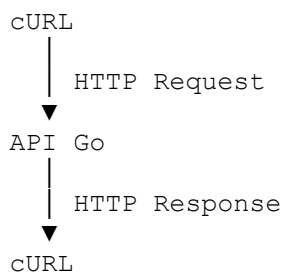
Com cURL podemos testar chamadas como:

```
GET
POST
PUT
PATCH
DELETE
```

Por exemplo:

```
curl.exe "http://localhost:8080/produtos"
```

Nesse caso, o cURL funciona como um **cliente HTTP**:



Isso é muito útil porque conseguimos testar uma API mesmo sem ter um frontend.

2. cURL no Windows

Versões modernas do Windows normalmente já possuem o `curl.exe`.

Para verificar, abra o CMD ou PowerShell e execute:

```
curl.exe --version
```

Se estiver instalado, aparecerão informações sobre a versão:

```
curl 8.x.x
Protocols: http https ...
```

Também podemos descobrir onde ele está instalado:

```
where curl
```

Normalmente:

```
C:\Windows\System32\curl.exe
```

Caso não esteja disponível, uma possibilidade é instalar com o WinGet:

```
winget install cURL.cURL
```

3. Cuidado: curl no PowerShell

Este é um ponto importante.

Dependendo da versão do **Windows PowerShell**, escrever:

```
curl
```

pode chamar um alias relacionado ao:

```
Invoke-WebRequest
```

em vez do programa `curl.exe`.

Por isso, durante nossas aulas, vamos preferir explicitamente:

```
curl.exe
```

em vez de simplesmente:

```
curl
```

Assim sabemos exatamente qual programa está sendo executado.

Podemos verificar o significado de `curl` com:

```
Get-Command curl
```

E verificar o executável:

```
Get-Command curl.exe
```

4. Nossa API de exemplo

Vamos imaginar que nossa API Go esteja funcionando em:

```
http://localhost:8080
```

E possua os seguintes endpoints:

```
GET      /produtos
```

```
POST     /produtos
```

```
GET      /produto?id=1
```

```
PATCH    /produto?id=1
```

```
DELETE   /produto?id=1
```

Antes de testar, execute o programa Go:

```
go run main.go
```

Você deverá receber algo como:

```
Servidor em http://localhost:8080
```

Mantenha esse terminal aberto.

Abra **outro PowerShell** para realizar as chamadas com cURL.

5. Fazendo uma chamada GET

GET é o método utilizado normalmente para consultar informações.

Execute:

```
curl.exe "http://localhost:8080/produtos"
```

Podemos receber:

```
[
  {
    "id": 1,
    "nome": "Notebook",
    "preco": 4500
  },
  {
    "id": 2,
    "nome": "Mouse",
    "preco": 120
  }
]
```

Não precisamos informar `-X GET`, porque GET é o método padrão do cURL.

Portanto:

```
curl.exe "http://localhost:8080/produtos"
```

já realiza um GET.

6. Visualizando os headers

Podemos utilizar:

```
-i
```

para visualizar também os headers da resposta.

Exemplo:

```
curl.exe -i "http://localhost:8080/produtos"
```

Podemos receber:

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Mon, 10 Aug 2026 18:00:00 GMT

[{"id":1,"nome":"Notebook","preco":4500}]
```

Agora conseguimos observar três elementos:

```
STATUS CODE
```

```
HTTP/1.1 200 OK
```

HEADER

```
Content-Type: application/json
```

BODY

```
[{"id":1,"nome":"Notebook","preco":4500}]
```

Isso torna -i muito interessante para estudar APIs.

7. GET utilizando Query Parameters

Nossa API permite procurar um produto usando:

```
/produto?id=2
```

Faça:

```
curl.exe -i "http://localhost:8080/produto?id=2"
```

O trecho:

```
?id=2
```

é um **Query Parameter**.

No Go podemos capturá-lo com:

```
idTexto := r.URL.Query().Get("id")
```

Resposta:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Body:

```
{
  "id": 2,
  "nome": "Mouse",
  "preco": 120
}
```

8. Testando um erro 404

Podemos solicitar um produto que não existe:

```
curl.exe -i "http://localhost:8080/produto?id=999"
```

Resposta:

```
HTTP/1.1 404 Not Found
Content-Type: application/json
```

Body:

```
{
  "erro": "Produto não encontrado"
}
```

```
}
```

Esse é um excelente teste para entender que uma response HTTP não possui somente o JSON. Ela também possui um **status code**.

9. Enviando dados com POST

POST normalmente é utilizado para cadastrar um novo recurso.

Nossa API espera algo como:

```
{
  "nome": "Webcam",
  "preco": 299.90
}
```

Precisamos informar três coisas ao cURL:

Método

↓

POST

Header

↓

Content-Type: application/json

Body

↓

```
{"nome": "Webcam", "preco": 299.90}
```

É aqui que precisamos tomar mais cuidado no PowerShell.

10. O problema das aspas no PowerShell

JSON utiliza aspas duplas:

```
{
  "nome": "Webcam"
}
```

O shell também utiliza aspas para delimitar argumentos.

Isso pode provocar problemas ao escrever comandos diretamente no terminal.

Um comando mal interpretado pode fazer o servidor receber algo como:

```
{nome:Webcam,preco:299.90}
```

Isso **não é JSON válido**.

JSON exige:

```
{
  "nome": "Webcam",
  "preco": 299.90
}
```

Por isso, quando um servidor Go retorna:

```
{
  "erro": "JSON inválido"
}
```

```
}
```

nem sempre significa que o problema está no código Go.

O problema pode estar na maneira como o terminal montou a chamada.

11. Forma segura para POST no PowerShell

Uma abordagem bastante clara para quem está começando é deixar o próprio PowerShell construir o JSON:

```
$body = @{  
    nome = "Webcam"  
    preco = 299.90  
} | ConvertTo-Json
```

Podemos visualizar o resultado:

```
$body
```

Depois fazemos a chamada:

```
Invoke-RestMethod `  
    -Uri "http://localhost:8080/produtos" `  
    -Method POST `  
    -ContentType "application/json" `  
    -Body $body
```

Observe que aqui estamos usando o **Invoke-RestMethod do PowerShell**, e não cURL.

Para aulas introdutórias no Windows, essa pode ser uma ótima alternativa para requisições JSON mais complexas.

12. POST com curl.exe

Também podemos utilizar o cURL verdadeiro.

Uma possibilidade é colocar o JSON em um arquivo, o que evita grande parte dos problemas de escape do terminal.

Crie:

```
produto.json
```

Conteúdo:

```
{  
    "nome": "Webcam",  
    "preco": 299.90  
}
```

Depois execute:

```
curl.exe -i -X POST "http://localhost:8080/produtos" -H "Content-Type:  
application/json" --data-binary "@produto.json"
```

Essa é uma maneira excelente para aulas porque o aluno consegue visualizar claramente o JSON enviado.

13. Entendendo o comando POST

Observe:

```
curl.exe -i -X POST "http://localhost:8080/produtos" -H "Content-Type: application/json" --data-binary "@produto.json"
```

Podemos dividir:

curl.exe

Executa o cURL.

-i

Mostra headers e status da resposta.

-X POST

Define o método POST.

"http://localhost:8080/produtos"

Define o endpoint.

-H "Content-Type: application/json"

Adiciona um header.

Estamos informando:

O Body que estou enviando contém JSON.

Finalmente:

--data-binary "@produto.json"

manda o conteúdo do arquivo como Body da request.

14. PATCH

PATCH normalmente é utilizado para alterar parcialmente um recurso.

Imagine:

```
{
  "id": 1,
  "nome": "Notebook",
  "preco": 4500
}
```

Queremos alterar somente:

```
{
  "preco": 3999.90
}
```

Crie:

patch.json

Com:

```
{  
  "preco": 3999.90  
}
```

Execute:

```
curl.exe -i -X PATCH "http://localhost:8080/produto?id=1" -H "Content-Type:  
application/json" --data-binary "@patch.json"
```

A API poderá retornar:

```
{  
  "id": 1,  
  "nome": "Notebook",  
  "preco": 3999.90  
}
```

15. DELETE

DELETE é mais simples porque, nesse exemplo, não precisamos enviar JSON.

Para excluir o produto 2:

```
curl.exe -i -X DELETE "http://localhost:8080/produto?id=2"
```

A API poderá responder:

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

Body:

```
{  
  "mensagem": "Produto removido com sucesso"  
}
```

Depois podemos conferir:

```
curl.exe "http://localhost:8080/produtos"
```

16. O modo Verbose

Uma opção muito útil para aprender e descobrir problemas é:

```
-v
```

Exemplo:

```
curl.exe -v "http://localhost:8080/produtos"
```

O cURL mostra mais detalhes da comunicação.

Você poderá encontrar informações sobre:

```
conexão  
request  
método  
headers enviados  
headers recebidos
```


response

É especialmente útil quando:

"A API não está funcionando e eu não sei por quê."

Experimente:

```
curl.exe -v "http://localhost:8080/produto?id=1"
```

17. Não copie URLs em formato Markdown

Outro erro muito comum acontece ao copiar links de páginas, chats ou documentos.

Isto está errado no terminal:

```
[http://localhost:8080/produtos] (http://localhost:8080/produtos)
```

Esse é um formato de **Markdown**, não uma URL para o cURL.

No terminal queremos somente:

```
http://localhost:8080/produtos
```

Portanto:

```
curl.exe "http://localhost:8080/produtos"
```

18. Quebra de linha no PowerShell

No PowerShell podemos usar o **backtick**:

```
`
```

para continuar um comando na próxima linha.

Exemplo:

```
Invoke-RestMethod `
    -Uri "http://localhost:8080/produtos" `
    -Method POST `
    -ContentType "application/json" `
    -Body $body
```

Cuidado: o backtick deve ficar no final da linha.

Para alunos iniciantes, se houver dúvida, mantenha comandos pequenos em uma única linha.

19. CMD e PowerShell não são iguais

É importante explicar aos alunos que:

```
CMD
PowerShell
Git Bash
WSL
```

são ambientes diferentes.

Um comando encontrado na internet para Linux ou Bash pode precisar de adaptações para PowerShell.

Por exemplo, a maneira como cada shell interpreta:

```
'  
"  
\  
`  
$
```

pode ser diferente.

Por isso um comando de cURL que funciona perfeitamente no Linux pode apresentar problemas quando simplesmente copiado para o PowerShell.

20. Resumo dos principais comandos

Objetivo	Comando
Ver versão	<code>curl.exe --version</code>
GET	<code>curl.exe "URL"</code>
Mostrar headers	<code>curl.exe -i "URL"</code>
Verbose	<code>curl.exe -v "URL"</code>
Definir método	<code>curl.exe -X POST "URL"</code>
Header	<code>-H "Content-Type: application/json"</code>
POST	<code>-X POST</code>
PATCH	<code>-X PATCH</code>
DELETE	<code>-X DELETE</code>

JSON em arquivo `--data-binary "@arquivo.json"`

Para nossa API Go, os quatro testes principais ficam:

```
# GET  
curl.exe -i "http://localhost:8080/produtos"  
  
# GET por ID  
curl.exe -i "http://localhost:8080/produto?id=1"  
  
# POST usando arquivo JSON  
curl.exe -i -X POST "http://localhost:8080/produtos" -H "Content-Type:  
application/json" --data-binary "@produto.json"  
  
# DELETE  
curl.exe -i -X DELETE "http://localhost:8080/produto?id=2"
```

E para PATCH:

```
curl.exe -i -X PATCH "http://localhost:8080/produto?id=1" -H "Content-Type:  
application/json" --data-binary "@patch.json"
```

A principal recomendação para os alunos no Windows é: **no PowerShell, prefira escrever `curl.exe`; tenha atenção especial às aspas do JSON; e, quando o JSON começar a ficar complexo, coloque-o em um arquivo `.json` ou use `ConvertTo-Json` com `Invoke-RestMethod`.** Isso evita grande parte dos erros que aparecem durante os testes de APIs.